Week 1: programs, text, math, variables and types

MIS41110 Programming for Analytics · 2026/27 · Week 1 lecture notes

These notes cover week 1's lecture, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

1 · What a program is, and why you

Slides, pages 4 to 7.

What a computer program is

A program is a **sequence of operations** for a computer to carry out, and a computer does nothing at all without one. **Hardware** is the physical part: the processor, the memory, the disks, the screen. On its own it is like your body without instructions: it is there, and it does nothing. **Software** is the instructions that make the hardware do something useful, and everything you write this term is software. Programs run far beyond laptops: phones, cameras, watches, light bulbs, washing machines, even coffee makers. Convenient for us: this term, we are the ones programming the café.

"But I'm a business student!"

Fair question. In analytics, programming is data analysis past what a spreadsheet can express, business software like stock control, and automation: a pipeline that takes a data set, cleans it, passes it on and processes it again, with no dragging and dropping by hand. If you get good, it is trading rules, which is your group project. And if you can do it yourself, you can also check whether someone else did it right, a colleague or an AI. There is a French saying I like: *on n'est jamais si bien servi que par soi-même*, if you want something done right, do it yourself. That is this module in one sentence.

Hardware, in one slide

Whatever the machine, its parts do three jobs: **input** gets data in (keyboard, microphone, network), **output** gets results out (screen, printer, speakers), and the **system** runs your program. The two parts you care about are the **CPU**, which does the work, and the **RAM**, which holds what it is working on. Your program is a guest of both, shown to its seat by the operating system. That is all the hardware we need.

Check yourself.

- **1.1** In one sentence each, what is hardware and what is software?

See also. py4e chapter 1, *Why should you learn to write programs?*

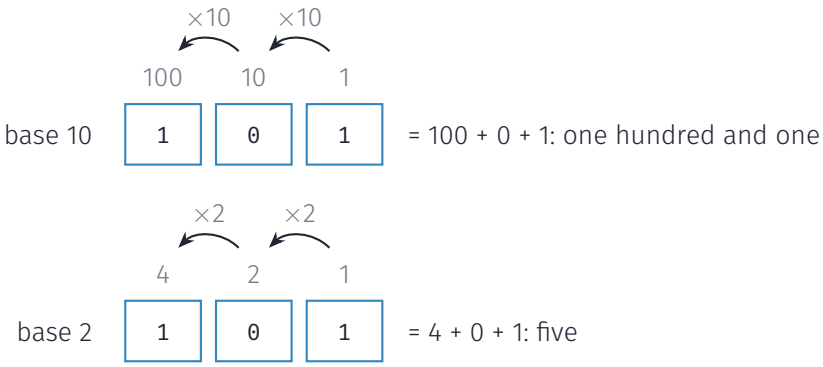
2 · How computers speak

Slides, pages 8 to 11.

Binary

Underneath, a computer has exactly one trick: a wire carries current, or it does not. Current is a 1, no current is a 0, and that is the whole alphabet. Counting with only two digits is **base 2**, or binary. In base 10, each position is worth ten times the one to its right: ones, tens, hundreds. In base 2, each position is worth twice the one to its right: 1, 2, 4, 8, 16, 32, reading from the right. So binary `10` is one 2 and zero 1s, which is two, and binary `101` is one 4, no 2 and one 1, which is five. Now the joke on page 8 works: there are 10 types of people in this world, those who understand binary and those who don't. Read in base 2, that `10` is two. George Boole worked out the logic of true-or-false values long before the computer existed, and it fits that on-or-off wire perfectly.

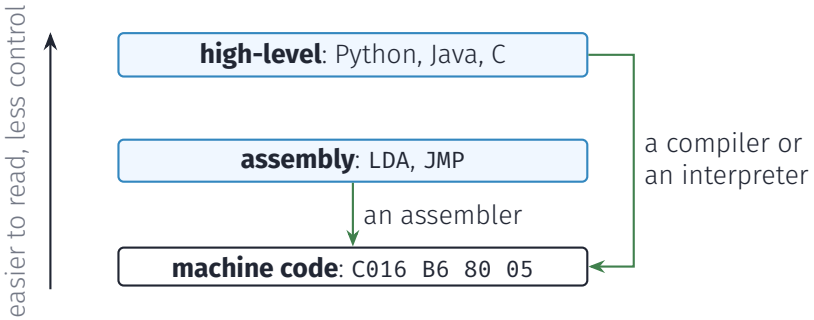
The same three digits, `101`, in both bases: the positions are worth 100, 10 and 1, or 4, 2 and 1, and that is the only difference.



From machine code to Python

How does a human write those 0s and 1s? We climbed a ladder. At the bottom is **machine language**: binary instructions specific to one processor, usually shown in hexadecimal, like `C016 B6 80 05`. Unreadable and unwritable by normal people. One rung up is **assembly**: short mnemonics like `LDA` (load a value) and `JMP` (jump to another place in memory), turned into binary by an **assembler**. Programs were once written this way, moving values around memory one instruction at a time. At the top are **high-level languages**, readable ones that a **compiler** or an **interpreter** translates for you: FORTRAN in the 1950s, then COBOL, C, and in the 1990s Python, Java and JavaScript. Each rung trades a little raw control for a lot of readability, and readability is where we live.

Here is the ladder, with what takes each upper rung down to machine code.



Which language for which job, and why Python

Every language was made for a reason. Assembly and C buy maximum speed at the price of comfort, which is how operating systems get built. C++, Java and C# build large structured applications. Swift makes iPhone apps; HTML and JavaScript make the web; R and Matlab do science. **Python** is general-purpose, compact, itself written in C, and the common language of data work. It has topped the popularity indexes for years, which matters to you in a practical way: more ready-made packages, more answers online when you are stuck at midnight, more jobs asking for it.

Python was created by Guido van Rossum to be easy to start and hard to outgrow, and it carries a small philosophy, the *Zen of Python*: beautiful is better than ugly, simple is better than complex, and **readability counts**. All term I call code *pythonic* when it is written the way the language wants to be written: clean rather than clever. Pythonic code reads like a short story you could hand to a friend, and the friend who matters most is you, three months from now, trying to remember what you did.

Common mistakes. Binary is not decimal with fewer digits: `10` in binary is two, not ten, because the second position from the right is worth 2.

Check yourself.

- **2.1** Which number does binary `110` stand for? And how do you write seven in binary?
- **2.2** Swift makes iPhone apps. Which rung is it on, what sits below it, and what takes it to the processor?

See also. py4e chapter 1. The exam can ask the theory: machine code against assembly against high-level languages.

3 · Your first program

Slides, pages 12 to 17.

The first line, live

Tradition says your first program prints `Hello, World!`. We keep the tradition and make it ours. Open a terminal, type `python3` (on Windows, `python`) to start the interpreter, and type one line:

```
>>> print("The Daily Grind is open for business.")
The Daily Grind is open for business.
```

The quotes are gone: Python printed what was between them, not the quote marks. That is your first program, and it works.

The same program, elsewhere

In C, the same output takes an `#include` line to load the input-and-output library, a `main` function, a `printf` call and a `return`. In Java you need a whole class, a `public static void main` method, and the printing buried three layers down. In assembly it is about fifteen lines; in machine code, a wall of hex. Four languages, the same output, wildly different amounts of ceremony. Ours was one line.

Anatomy of that one line

`print()` is a **function**: a small program Python already has, ready to use, which you call by name with parentheses. Python is **case sensitive**, so `print`, `Print` and `PRINT` are three different names, and only the first is the function. The text between the quotes is a **string literal**: a string is a chain of characters. Single or double quotes both work, as long as they match. And a quote of the other kind sits happily inside:

```
>>> print('She said "open"')
She said "open"
>>> print("It's open")
It's open
```

Inside single quotes, a double quote is just a character, and the other way around. Open with one kind and close with the other, and the string never ends:

```
>>> print('Open for business.")
SyntaxError: unterminated string literal (detected at line 1)
```

Three ways to run Python

1. **The interpreter**, or shell: type `python3` in a terminal and each line runs when you press Return. Its prompt is `>>>`. Quit with `quit()`, or Ctrl-D (on Windows, Ctrl-Z then Return). From Python 3.13 it may also indent the lines of a block for you: week 2's notes show how to type one. 2. **A file**: write the lines into a file ending in `.py`, say `cafe.py`, and run `python3 cafe.py` (on Windows, `python cafe.py`). Everything runs top to bottom. This is how our café runs. 3. **An IDE** like VS Code: editor and console in one window, with a run button.

The notation on my slides: `$` is the terminal, `>>>` is the Python shell, and plain code is a program file. Now put the same line in a new file, `cafe.py`, save it, and run it from the terminal:

```
$ python3 cafe.py
The Daily Grind is open for business.
```

You write it in the editor, you run it in the terminal, and the file stays on disk. That round trip, edit, save, run, is the loop you live in from now on.

Where does the shell save your code? Nowhere: everything you type at `>>>` is gone when you quit. To keep code, write it in a `.py` file.

The terminal also has to be **in the folder that holds your file**, or `python3 cafe.py` cannot find it. The easiest way: open the folder in VS Code (File, then Open Folder), then open its own terminal (Terminal, then New Terminal), which starts right there. On a Mac you can also type `cd ` (with the space), drag the folder from the Finder into the Terminal window to paste its path, and press Return. On Windows, right-click the folder and open a terminal there.

The shell and a file differ in one more way. In the shell, typing a value on its own shows it, and a string shows with its quotes, because the shell shows you the value itself. `print` shows the text:

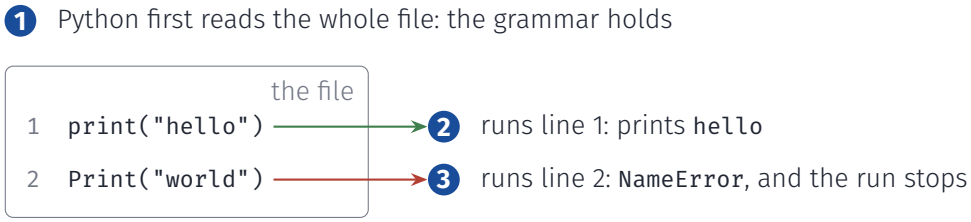
```
>>> "hello"
'hello'
>>> print("hello")
hello
```

A file shows only what you print: a line holding just `"hello"` shows nothing when the file runs.

Line by line, but checked first

Python is called an interpreted language: it runs your program line by line, where C compiles the whole program first. But Python does read your whole file before it runs any of it. If any line breaks the language's grammar, a **SyntaxError**, nothing runs at all, not even line 1. Other errors only show up while the program runs: Python runs line 1, then stops at the line that fails. With `print("hello")` on line 1 and `Print("world")` on line 2, the file prints `hello`, then stops on line 2 with a `NameError`.

That file, in the order Python works on it. ❶ It reads the whole file and checks the grammar: `Print("world")` is grammatical, only a name that does not exist, so nothing stops here. ❷ It runs line 1, and `hello` prints. ❸ It runs line 2, finds no `Print`, and stops with a `NameError`. Grammar is checked for the whole file before line 1 runs; everything else is found only when its line runs.



Your turn: print, break it, fix it

The first TRY IT, page 16: print the café's opening line, then break it on purpose with a capital P, read the error, and fix it. Here is the error you are hunting for:

```
>>> Print("The Daily Grind is open for business.")
NameError: name 'Print' is not defined. Did you mean: 'print'?
```

That is a `NameError`: the name `Print` does not exist, and a recent Python (3.10 or later) is kind enough to suggest the one that does. Read your errors; they are usually this helpful. And type the code yourself rather than copy it from the slides. The mistakes you make on the way, a missing letter, a parenthesis in the wrong place, a quote that never closes, are the lesson.

Every message in these notes is what Python 3.12 to 3.15 print, and where they differ, the notes say which version prints what. An older Python 3, like the 3.9 a Mac may already have as `python3`, words more of them differently and suggests nothing: one more reason to install the current version from python.org.

Comments

```
# Formal and polite
print("The Daily Grind is open for business.")
print("What can I get you?")   # a bit friendlier
```

Everything after `#` on a line is a **comment**: Python does not run it, does not print it, does not care. It is for the humans reading the code, and the most important one is you, three weeks from now. Comment **why**, not what: "add one to x" beside a line that adds one to x tells me nothing I cannot see. A `#` at the start of a line is also how you switch a line off while you hunt a bug, without deleting it.

> **Good practice.** If you already code in another language, take up Python's own style guide from day one: PEP 8. Four spaces per indentation level, `snake_case` for variables and functions, `UPPER_CASE` for constants, spaces around operators, and short lines (PEP 8 says 79 characters, and lets a team agree on up to 99; the café's files run to about 90). None of it changes what the

> program does; all of it changes how fast someone else, or you next month, can read it. Most Python code you will meet follows it, so code that follows it reads as familiar.

Common mistakes. `print` with a capital letter is a `NameError`. Mismatched quotes are a `SyntaxError`. And running `python3 cafe.py` from the wrong folder fails before Python reads a single line of your code: move the terminal to the file's folder first.

Check yourself.

- **3.1** Why does `print('It's open')` fail, and how do you fix it by changing the outer quotes?
- **3.2** A file holds these two lines, the second with its parenthesis never closed. What does running the file print?

```
print("ready")
print("set"
```

See also. Sheet 1, exercise 0, the setup gate. The Function reference, Week 1: `print`. py4e's *Installing Python*.

4 · Under the hood

Slides, pages 18 and 19.

Everything is an object

```
>>> print(type("The Daily Grind"))
<class 'str'>
```

In Python **everything is an object**: text, numbers, the result of a calculation. Every object has a **type**, and `type()` tells you which one you are holding. `str` is Python's name for a string. This one function answers about half of your "why does this not work?" questions all term: when in doubt, `type()` it. Its partner is `help()`. `help(str)` shows everything a string can do, and `help(print)` shows that `print` takes more than the text: a separator, an `end`, and a few others. In a terminal it opens one screen at a time: the arrow keys scroll, and `q` takes you back to `>>>`. Two ways to ask Python about Python, no web search needed.

What a string really is

Everything in a computer is a number, and every number is binary. So what is a string, underneath? A sequence of **character codes**. `"Hello, World!"` is the characters H, e, l, l, o and so on; their codes in decimal are 72, 101, 108, 108, 111; the same codes in hexadecimal are 48, 65, 6c, 6c, 6f; and in binary, H is `0100 1000`, e is `0110 0101`, l is `0110 1100` and o is `0110 1111`. Same information, four spellings, and Python skips all of it for you. The lookup table is called ASCII, and it is worth a two-minute search. This is examinable: what is a string, underneath? A sequence of character codes. Nothing more.

Each character is one code, written four ways. Read a column down: the `H` you see is 72 in decimal, 48 in hexadecimal and 0100 1000 in binary, the 0s and 1s the wire carries.

the first five characters of `"Hello, World!"`

what you see	`H`	`e`	`l`	`l`	`o`
decimal	72	101	108	108	111
hexadecimal	48	65	6c	6c	6f
binary	0100 1000	0110 0101	0110 1100	0110 1100	0110 1111

and so on, to `!`

Check yourself.

- **4.1** What does `type("3.5")` say, and why is it not a number?

See also. The Function reference, Week 1: `type` and `help`.

5 · Working with text

Slides, pages 20 to 24.

What a receipt holds

Poll. The break slide, page 20, asks: what pieces of information must a receipt hold? A receipt can hold the items, the price and quantity of each, the total, the taxes, a discount, the shop's name, even the name of whoever served. The core is the items, their quantities and prices, and the total computed from them. Section 10 turns each piece into a variable and prints the receipt; this section builds the text tools it prints with.

Special characters

What if the text itself contains the quote that wraps it? Python needs a hint, and the hint is the **backslash**, the escape character: it gives the next character a special meaning.

```
>>> print("How's it goin', \"World\"?")
How's it goin', "World"?
>>> print('Ready?\n\tYes.\n\tNo.')
Ready?
    Yes.
    No.
```

`\"` and `\'` are literal quotes that do not end the string. `\n` is a new line and `\t` is a tab: it jumps to the next tab stop, a fixed column every eight characters in most terminals, which is why it lines things up where spaces would not. And `\\` is a real backslash. Python does exactly what you write and nothing more: a space before or after `\n` is printed too. It does not tidy your text the way a chatbot would.

When the quotes collide and nothing escapes them, Python stops before running anything:

```
>>> print("say "hi" now")
SyntaxError: invalid syntax. Is this intended to be part of the string?
```

The first `"` opens a string, the second closes it, and `hi` sits outside any string, so the line makes no sense to Python. Python 3.14 and later guess right, that `hi` belongs in the string; Python 3.12 and 3.13 guess `Perhaps you forgot a comma?`, which is wrong here. The fix is `\"hi\"`, or single quotes around the whole text:

```
>>> print("say \"hi\" now")
say "hi" now
```

Python pairs quotes from the left. (1) In the broken line, the first `"` opens a string and the second closes it, so `hi` lands outside any string, and the third `"` opens a second string, `" now"`. (2) In the fixed line, each `\"` is one character inside the string, so the string runs from the first quote to the last, and `print` shows the quotes.

(1) `print("say "hi" now")` — two strings, and `hi` outside both: `SyntaxError`

(2) `print("say \"hi\" now")` — one string; each `\"` is one character

Triple quotes

Between triple quotes, `'''` or `"""`, a string can span lines and hold quotes without escapes:

```
>>> print('''I'm ok.
... I think.''')
I'm ok.
I think.
```

The `...` is the shell telling you it knows the statement is not finished: the string is still open, or the parenthesis of `print` is. It keeps waiting until you close both. Escaped, double-quoted or triple-quoted, it is the same string inside the computer and it prints the same way: the quotes are how you write the text, not what the text is. Escapes still work inside triple quotes: a `\t` there is still a tab. And for a long line of code, a backslash as its very last character continues it on the next line: handy now and then, easy to overuse, so use it sparingly.

A trick you will see in other people's code: a block of lines wrapped in triple quotes inside a file does not run either. It is not a real comment. Python reads it as a string and throws it away, and in the shell it would print it back. For comments, use `#`.

String arithmetic

Strings have operators too. `+` joins, `*` repeats, `len()` measures:

```
>>> print("The Daily " + "Grind")
The Daily Grind
>>> print("=" * 34)
==================================
>>> print(len("The Daily Grind"))
15
```

`+` glues exactly what you give it: no space is added unless you add one. `"=" * 34` is our receipt divider: one character, multiplied, instead of typing thirty-four equals signs like an animal. And `len()` counts every character, spaces included. You can go further:

```
>>> print("he" + "l" * 2 + "o")
hello
```

`*` goes before `+` for text exactly as it does for numbers, so the two `l`s are made first.

Predict, then run

The TRY IT on page 24 asked for six predictions:

```
print("To break a line, use \\n.")
print("First line."); print("Second line.")
print("First line.\n" + "Second line.")
print("Decaf? Yes.\b\b\b\bNO!!")
print("G" + "o" * 10 + "d morning, Dublin!")
print("Single", end=" "); print("line.")
```

In a terminal, they show:

```
To break a line, use \n.
First line.
Second line.
First line.
Second line.
Decaf? NO!!
Gooooooooood morning, Dublin!
Single line.
```

`\\` prints a real backslash, so the `n` after it is just a letter. The semicolon puts two statements on one line: legal, and almost always a bad idea. `\b` is a backspace: each one moves the cursor back one place, so `NO!!` writes over `Yes.` on the screen. `"o" * 10` makes ten `o`s between the `G` and the `d`. And `end=" "` changes what `print` puts at the end of its line, a new line by default, so the second `print` carries on the same line. In the shell you can see it happen: after a `print` whose `end` is a space, the `>>>` prompt comes back on the same line as the text.

Common mistakes. A quote of the same kind inside a string ends it early: escape it or use the other kind outside. And `+` adds no space: `"Hello" + "World"` is `HelloWorld`.

Check yourself.

- **5.1** How many characters does `len("Tab\there")` count, and why?
- **5.2** A file holds two lines, `print("Hi", end="!")` and then `print("Bye")`. What does running it show?
- **5.3** Fix `print('It's open')` without touching its outer quotes.

See also. Sheet 1, exercise 2, the door sign. The Function reference, Week 1: `print`, its `end`, and `len`.

6 · Math

Slides, pages 25 to 28.

The operators

```
>>> print(2 + 5, 2 - 5, 2 * 5, 2 / 5, 2 % 5)
7 -3 10 0.4 2
```

`print` takes several values separated by commas and prints them with a space between. `%` is the **remainder**: what is left after you take out as many whole copies of the second number as fit. `//` is **floor division**, the division rounded down:

```
>>> print(150 % 5, 151 % 5, 151 // 5)
0 1 30
```

150 is exactly thirty 5s, so nothing is left; 151 leaves one over. The remainder is also the instant even-or-odd test: a number `% 2` is 0 when it is even and 1 when it is odd.

A single / always gives a float, even when the division comes out exact:

```
>>> print(150 / 5)
30.0
```

Powers use two stars: `2 ** 5` is 32. Excel, calculators and other languages write powers with a caret, ^, but in Python ^ is a different operator altogether, so `2 ^ 5` gives 7, not 32. Always `**` for powers, with no space inside: `2 * * 5` is a `SyntaxError`. And put spaces around your operators: Python ignores them, humans do not, and spaced-out math is the pythonic way.

Good practice. Coming from C or Java? Python's `//` rounds down, toward minus infinity, where those languages cut toward zero, and `%` follows it:

```
>>> -7 // 2
-4
>>> -7 % 2
1
```

C and Java, which write the first one `-7 / 2`, give -3 and -1. So a parity test written `n % 2 == 1` works for negative numbers in Python and fails in C. When you move arithmetic from one language to another, test a negative input.

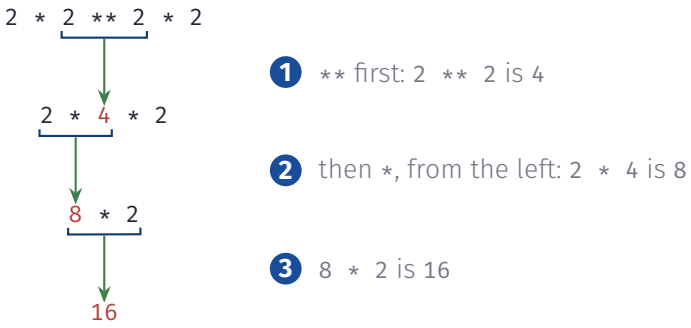
Precedence, by hand

Page 26 is a TRACE: work each line out on paper before Python does, because in the exam Python is not in the room.

Expression	Python says	Why
`2 + 3 * 4`	`14`	`*` before `+`
`(2 + 3) * 4`	`20`	parentheses first
`2 - - 1`	`3`	minus a negative
`2 - - - 1`	`1`	three minuses
`2 * 2 ** 2 * 2`	`16`	`**` binds tightest
`3 / 2` against `3 // 2`	`1.5` against `1`	division, then floor division

Python follows the order of operations from your math classes: parentheses, then powers, then multiplication and division, then addition and subtraction. And `2 - - 1` is 3 because the two minus signs multiply into a plus. When in doubt, use parentheses: they cost nothing and settle every argument about what happens first.

Here is the table's fifth row worked one operation at a time, the way to trace any expression on paper: each step does one operation and writes the line again with the result in its place. (1) `**` binds tightest, so `2 ** 2` goes first and becomes 4. (2) That leaves two multiplications, done from the left: `2 * 4` is 8. (3) Then `8 * 2` is 16. Worked strictly from the left, the same line would give 32: the operators decide what goes first, and reading order only settles a tie between equals, like the two `*`.



Division by zero

```
>>> print(2 / 0)
Traceback (most recent call last):
ZeroDivisionError: division by zero
```

(Between those two lines, Python also says where the error happened; your shell prints it too.)

Python stops your program. Zero denominators hide in innocent formulas: the café's average sale per order, at opening time, when there are no orders yet, is revenue divided by zero, and the register dies. When you write a calculation, ask what happens on the first day, when every count is still zero. The café meets this for real in week 5.

The café does its math

```
espresso_price = 2.50
muffin_price = 2.40

# a customer buys 2 espressos and 1 muffin
total = espresso_price * 2 + muffin_price * 1
print(total)                    # 7.4
```

Prices, quantities, a total: arithmetic is the register's day job. In the shell, the three assignment lines print nothing at all: each stores a value, and only `print(total)` shows anything. The names on the left are variables, which section 9 introduces properly. The math came first because the math is why the register exists.

Common mistakes. `^` is not a power. `/` gives a float even for `150 / 5`. And a division whose bottom number can be zero will, one day, be zero.

Check yourself.

- **6.1** Muffins come in boxes of 12, and the café has baked 30. Write the two expressions that give the number of full boxes and the number of muffins left over, and their values.
- **6.2** A report divides the day's revenue by the number of orders. What happens on a day with no orders, and what is the error called?

See also. Sheet 1, exercises 1 and 4. The Function reference, Week 1: the operators, `//`, `%` and `**`. py4e chapter 2.

7 · Floats, and the wall

Slides, pages 29 to 32.

Floats

Decimals are **floats**, short for floating-point. They do arithmetic like integers, and mixing the two always gives a float: `2.0 + 5.0` is `7.0`, and `2 - 5.0` is `-3.0`. `type()` confirms it: `type(2 + 5)` is `int`, `type(2.0 + 5.0)` is `float`. But floats hide something, and it comes straight from that binary wire. A decimal number must be stored as 0s and 1s in a fixed format, called IEEE 754: a sign, a fraction and an exponent. Like recording music digitally: almost perfect, but not exact.

```
>>> print(10 / 3)
3.3333333333333335
```

About seventeen significant figures, then the format runs out of room. Memory is finite; the 3s cannot go on forever.

The wall: our first bug

A customer orders three lattes at 3.20 each. Nine-sixty, surely. Ask Python:

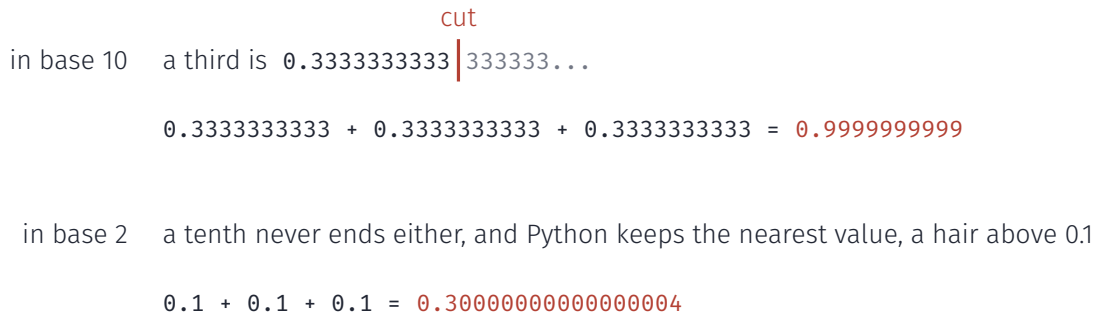
```
>>> print(3.20 * 3)
9.600000000000001
```

That is not a typo, and I did not do it on purpose. The canonical demonstration:

```
>>> print(0.1 + 0.1)
0.2
>>> print(0.1 + 0.1 + 0.1)
0.30000000000000004
```

Two tenths are fine, three are not. One tenth has no exact form in binary, the way one third has no exact form in decimal, so tiny errors creep in and pile up. Halves, quarters and eighths store exactly; tenths do not. And the errors can also cancel: `10/3 + 10/3 + 10/3` gives a clean `10.0`. So the rule is not that floats are broken. The rule is: **never depend on a float being exactly some precise value.**

The same thing happens in base 10 with a third. A ten-digit calculator shows a third as 0.3333333333 and cuts the rest, so three of them make 0.9999999999, not 1. A tenth has the same trouble in base 2: its digits never end, so Python keeps the nearest value it can store, a hair above 0.1, and three of them add up to 0.30000000000000004.



So is our register broken?

No. Our display is naive, which is a different thing. The stored value is off by about a millionth of a billionth of a euro, so nobody's coffee money is harmed. Every calculator rounds what it shows you. Where it can matter is a program that compares numbers to make a decision: a trading rule that buys above one price and sells below another can tip the wrong way on a difference that small. Next week brings formatting, and the receipt reads `9.60` like a grown-up register. Until then, The Daily Grind prints ugly totals on purpose, as a souvenir. Float imprecision is a classic exam question, and you can now answer it in one sentence.

Good practice. The café keeps its prices in floats on purpose, to show you the wall. A real system counts money in integer cents, which stay exact, and formats them only to print; financial code that needs decimals uses the `decimal` module, which is outside this course's toolbox and not on the project's library list. To compare two floats, ask whether they are close, never whether they are equal. `math.isclose` does it; the slides do not cover it, and `math` is on the project's library list:

```
>>> import math
>>> 0.1 + 0.1 + 0.1 == 0.3
False
>>> math.isclose(0.1 + 0.1 + 0.1, 0.3)
True
>>> 320 * 3                       # three lattes, in cents
960
```

In the project. Your group's desk keeps money for a fictional fund, and the Project rules mark its accounting to the cent: this section's wall follows you there. Week 2's formatting, and never testing two floats with `==`, are how you get past it. The integer cents of the box above are for real registers: how your desk holds its money is set by the project documents.

How big? How small?

Integers in Python are **unbounded**: `1000 ** 103`, a 1 with 309 zeros, prints without complaint. Floats are not, because their binary format has fixed limits. `1000.0 ** 102` is fine, shown in scientific notation as `1e+306`. One power more overflows:

```
>>> print(1000.0 ** 103)
OverflowError: (34, 'Result too large')
```

The other way, `0.1 ** 323` is `1e-323`, almost nothing but still there, and `0.1 ** 324` quietly **underflows** to `0.0`: no error, no warning. The point to carry away: **an int is exact and unlimited; a float is fast and approximate.** Neither is better. Choose knowingly.

Common mistakes. Comparing a float result with an exact value: `0.1 + 0.1 + 0.1` is not `0.3`. And trusting a long float tail on a receipt: it is the display, not the money.

Check yourself.

- **7.1** Is `0.5 + 0.25` exact in Python? Why, when `0.1 + 0.1 + 0.1` is not?
- **7.2** One of `10 ** 400` and `10.0 ** 400` raises an error. Which one, which error, and why not the other?
- **7.3** What does `type(3 + 0.5)` say, and why?

See also. Sheet 1, exercise 3. py4e chapter 2.

8 · Booleans

Slides, pages 33 and 34.

True and False

The café is either open or it is not, and that yes-or-no has its own type, the **boolean**, named after George Boole: exactly two values, `True` and `False`, capitalized.

```
>>> open_sign = True
>>> print(type(open_sign))
<class 'bool'>
>>> print(type(true))
NameError: name 'true' is not defined. Did you mean: 'True'?
```

Lowercase `true` is not a boolean: it is just a name nobody has defined. Worse, nothing stops you from defining it: after `true = "potato"`, the line `open_sign = true` makes the café's open sign a potato. `True` itself is protected:

```
>>> True = "potato"
SyntaxError: cannot assign to True
```

Booleans are secretly numbers

`True` behaves as 1 and `False` as 0 in arithmetic:

```
>>> print(False + False, True * True, False - True)
0 1 -1
>>> print(True / False)
ZeroDivisionError: division by zero
```

`True / False` is 1 divided by 0, our friend from the math section. Handy once in a while, for counting how many things are true, and a trap the rest of the time.

One more look-alike: `print(True)` and `print("True")` both show `True`, but one is a boolean and the other a string of four characters. Printing hides the difference; `type()` shows it.

The bool() quiz

Anything can be converted to a boolean with `bool()`. Page 34's seven predictions, run:

```
>>> print(bool(0), bool(1000000), bool(0.001), bool(-0.001))
False True True True
>>> print(bool("Hello!"), bool("False"), bool(""))
True True False
```

One rule decides all of them: **zero and empty are `False`; everything else is `True`.** A negative number is not zero, so it is `True`. And the trap: `bool("False")` is `True`, because `"False"` in quotes is a non-empty string. The text False is not the value `False`. It is exactly the kind of line the exam asks you to predict.

Page 34's seven values, sorted into the rule's two bins:



Good practice. Experienced Python programmers lean on that rule: `if name:` rather than `if name != "":` (the `if` arrives in week 3). Use it when empty or zero really means "no". When zero is a valid value, a quantity of 0 or a price of 0.0, spell the test out, because `if qty:` treats a real 0 as missing. The café spells the test out, `!= ""`, so it reads in the words you know; both are correct.

Common mistakes. `true` and `false` in lowercase are names, not booleans. And a string that spells a boolean is still a string.

Check yourself.

- **8.1** What do `bool(" ")`, a single space, and `bool("0")` give?
- **8.2** Why is `True = 1` refused when `true = 1` is accepted?

See also. The Function reference, Week 1: `bool`.

9 · Variables

Slides, pages 35 to 38.

A name tag on a value

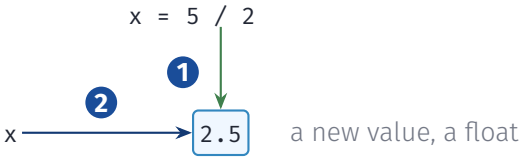
Poll. One word: what does a variable hold? A value, data, or a kind of data, a string, an integer, a float, characters: all of these are right. A variable holds a value, and values are data of different types, the type `type()` reports.

```
>>> x = 5
>>> print(x)
5
```

A variable **attaches a name to a value**, like the memory on a calculator: you store a number once and recall it later with MR. An assignment has two sides, and **the right-hand side is computed first**; then the name is attached to the result. So `x = 5 / 2` does the division, gets 2.5, and points `x` at it. Python works out the type for you: `x` is now a float, without you saying so. Variables hold text too:

```
>>> a_laugh = "ha"
>>> intensity = 3
>>> print(a_laugh * intensity)
hahaha
```

Here is `x = 5 / 2` in that order. ❶ Python computes the right-hand side first: the division makes a new value, 2.5, a float. ❷ Then it attaches the name `x` to it, the way a tag hangs on an item. The name is not the value: it points at it, and the next sections lean on that difference.



Naming: rules, then taste

The rules: letters, digits and _; a name may not start with a digit; and it may not be one of the 35 **keywords**, the words Python keeps for itself (page 36 lists them). `while = 2` is a `SyntaxError`. `While = 2`, with a capital W, is legal, and so is `Print = 12`: a perfect way to confuse the next person who reads your code, so please don't.

Then taste, the part humans read. Say you compute the circumference of a circle of radius 2, with pi stored as `x = 3.14`. `x * 2 * 2` works, and two months later nobody, you included, knows what it means. Named values explain themselves:

```
>>> pi = 3.14
>>> radius = 2
>>> print(2 * pi * radius)
12.56
```

A bare `x` tells the reader nothing; `pi` tells them everything. A name like `variable` is useless, and `the_value_of_pi_which_is_3_dot_14` is a crime of its own. In our café it is `espresso_price`, not `ep`, and `open_sign`, not `flag2`. A good name is the cheapest documentation you will ever write, and unlike a comment it cannot fall out of date.

Good practice. Never name a variable after a built-in. `str`, `list`, `sum`, `max`, `input` and `type` are all legal names, and assigning one hides the built-in for the rest of the file:

```
>>> str = "hello"
>>> str(5)
TypeError: 'str' object is not callable
>>> del str                        # the built-in is back
>>> str(5)
'5'
```

A name that says what the value is, `prices` or `order_total`, never collides.

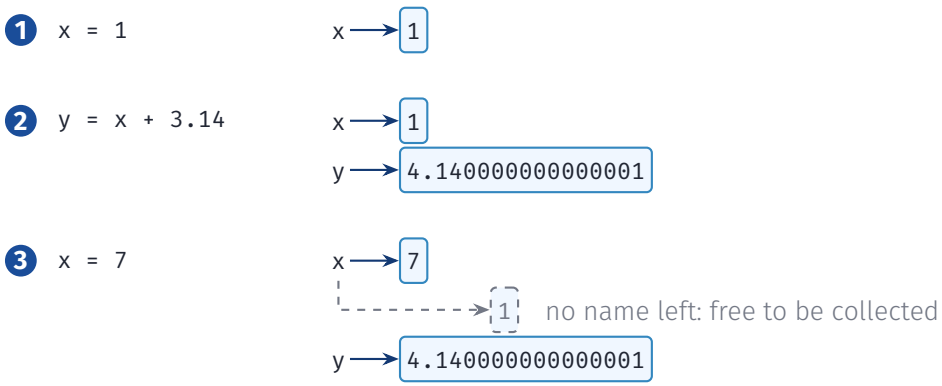
Rebinding: follow the arrows

Page 37 is a TRACE. What does this print?

```
x = 1
y = x + 3.14
x = 7
print(y)
```

The tempting answer is 10.14, using the new `x`. It prints `4.140000000000001`, the float wall again, because line 2 computed `y` from the value `x` had at that moment, which was 1. Rebinding `x` to 7 afterward does not reach back in time and change `y`: `y` holds its own value, and it does not stay wired to `x`. And when `x = 7` replaces `x = 1`, nothing in your program points at the old 1 any more, so Python is free to reclaim it: **garbage collection**, a service you get for free.

Follow the arrows line by line. ❶ `x = 1` puts the name `x` on the value 1. ❷ `y = x + 3.14` works out the right-hand side first, with the 1 that `x` points at then, and puts `y` on the result. ❸ `x = 7` moves `x` alone: `y` keeps its own value, and the 1, with no name left on it, is free to be collected.



You can also remove a name yourself with `del`:

```
>>> x = 7
>>> del x
>>> print(x)
NameError: name 'x' is not defined
```

Updating in place

Adding to a running total is so common that Python gives it its own operator:

```
revenue = 0.0
revenue += 7.40      # same as: revenue = revenue + 7.40
```

The whole family: `+= -= *= /= **= %= //=`. Each one takes the variable, applies the operation with the value on the right, and stores the result back: with `x = 4`, `x += 4` makes it 8, and doing it again makes it 12. Week 1's receipt ends with `revenue += total`. You will read `+=` in every Python program you ever open; our own café then writes the step out in full, `revenue = revenue + paid`, so a trace can show both sides.

Common mistakes. Expecting a variable to follow another after a rebinding. Naming a variable after a keyword. And printing a name after `del`, which is a `NameError`.

Check yourself.

- **9.1** After `price = 2.50`, `qty = 4` and `total = price * qty`, the line `qty = 5` runs. What is `total` now?
- **9.2** Which of these names are legal: `2nd_price`, `price_2`, `class`, `Class`?

See also. py4e chapter 2. The Function reference, Week 1: `X += VALUE`.

10 · Types and the receipt

Slides, pages 39 to 43.

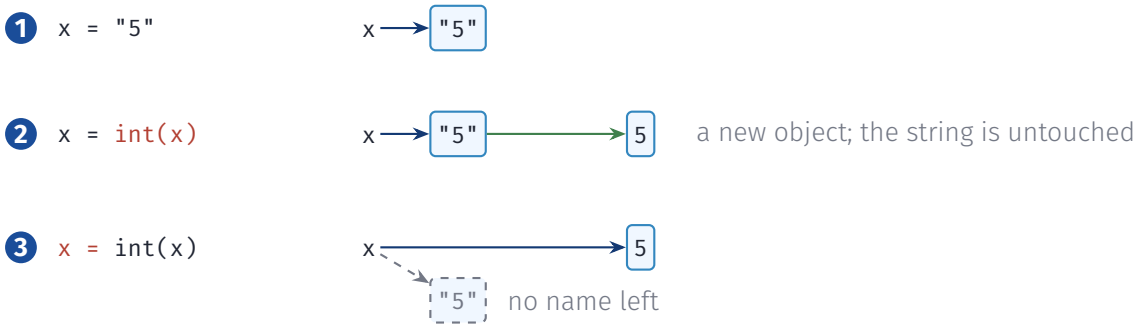
Types and conversions

Four types so far: `"5"` is a `str`, `5` is an `int`, `3.14` is a `float`, `False` is a `bool`. Each has a conversion function with the same name: `str()`, `int()`, `float()`, `bool()`. You will also hear converting called **casting**.

```
>>> x = "5"
>>> x = int(x)        # "5" (text) -> 5 (number)
```

Careful: `int(x)` does not change what you pass in. It builds a new object and hands it back. Above, `x = int(x)` rebinds `x` to it; skip the assignment and the new value is gone. And `print(int(3.14))` shows `3`: `int` cuts the decimal part off, it does not round, and the 3.14 itself is untouched.

1 `x = "5"` puts `x` on a string. Then `x = int(x)`, in two steps. 2 The right side runs first: `int(x)` builds a new object, the number 5, and leaves the string as it was. 3 Only then does the assignment move `x` onto the new number, and the string, with no name left, is free to be collected. In a file, `int(x)` on its own line builds a 5 that no name keeps.



The one you will hit first

```
>>> x = 100
>>> print("I am " + x + " years old.")
TypeError: can only concatenate str (not "int") to str
```

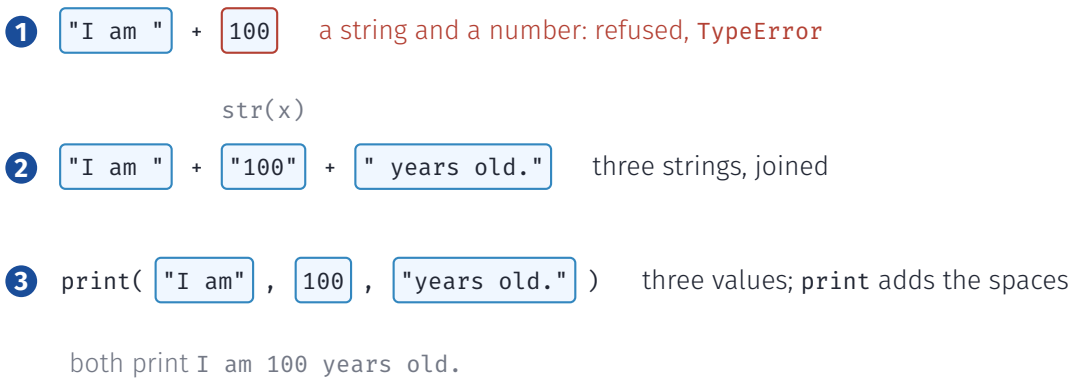
`+` between a string and a number is ambiguous: join the text, or do arithmetic? Rather than guess, Python stops and asks you to say which. The fix is to convert the number first:

```
>>> print("I am " + str(x) + " years old.")
I am 100 years old.
```

The refusal comes from `+`, not from `print`. `print` accepts any value, and with commas it prints several, a space between each:

```
>>> print("I am", x, "years old.")
I am 100 years old.
```

The three lines side by side. 1 `"I am " + x` meets a string on one side and a number on the other, and `+` will not guess between joining and adding. 2 `str(x)` turns the number 100 into the text `"100"`, so `+` joins three strings. 3 With commas, `print` takes three separate values, turns each into text itself, and puts a space between them.



This is the key to our receipt: every number that goes into text joined with `+` passes through `str()` first. Next week brings a much nicer way.

Good practice. Gluing text with `+` and `str()` is week 1's tool, and the receipt uses it on purpose this week. From week 2 on, write an f-string: `f"I am {x} years old."` converts for you, cannot hit this `TypeError`, and shows the sentence as it will print.

The receipt: the ask and the plan

The owner's words: *"Print me a receipt: the shop's name, each item with quantity and line total, the total, and whether we're open."* Decompose before you type. We have no loops and no functions yet, only top-to-bottom code, so three steps: variables for the name and the prices, arithmetic for the totals, strings for the printing. One design choice out loud: prices are floats. We know the wall is right there, and we accept it knowingly until week 2 fixes the display. Choosing with your eyes open is the skill.

The receipt: build

Here is the whole file, `cafe_w1.py`, top to bottom:

```
cafe_name = "The Daily Grind"
open_sign = True

espresso_price = 2.50
latte_price = 3.20
muffin_price = 2.40

espresso_qty = 2
muffin_qty = 1
total = espresso_price * espresso_qty + muffin_price * muffin_qty

print("=" * 34)
print(cafe_name + ": receipt")
print("=" * 34)
print("espresso  x " + str(espresso_qty) + "    " + str(espresso_price * espresso_qty))
print("muffin    x " + str(muffin_qty) + "    " + str(muffin_price * muffin_qty))
print("TOTAL: " + str(total))
print("Open: " + str(open_sign))

revenue = 0.0
revenue += total
print("Revenue so far: " + str(revenue))

print(type(cafe_name))
print(type(espresso_qty))
print(type(total))
print(type(open_sign))

latte_total = latte_price * 3
print("3 lattes: " + str(latte_total))
```

The names read like the receipt itself, which is the point of choosing them well. An editor's AI assistant will offer to finish the total line for you, and its suggestion can leave the quantities out entirely: plausible, and wrong. Read what a tool suggests before you accept it: that reading is the skill this module builds.

Test and reflect

Run it with `python3 cafe_w1.py`, from the folder that holds it:

```
==================================
The Daily Grind: receipt
==================================
espresso  x 2   5.0
muffin    x 1   2.4
TOTAL: 7.4
Open: True
Revenue so far: 7.4
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
3 lattes: 9.600000000000001
```

It works: a name, two lines with quantities and line totals, a total, the open sign, and the three-latte souvenir at the bottom. Now feel exactly how limited it is, because every limit is a lesson already on the calendar. The line totals print as `5.0` and `2.4`, not to the cent: formatting, week 2. The order is hard-coded, so no customer can say what they want: input, week 2. Three lattes print that tail, and we know why. And every new product means copying another print line: data structures, week 4. Four limitations, four coming attractions. That is how this course moves: we build the simplest thing that works, feel where it hurts, and the next tool is shaped like the pain.

The file is yours. Run it, then change it: a divider of stars instead of equals signs, a third product, a different order. Break it and fix it. That is how it becomes your code.

Common mistakes. Joining a number to text with `+` without `str()`. Converting without assigning, so the result is lost. And expecting `int()` to round: `int(2.9)` is 2.

Check yourself.

- **10.1** What does `print(str(5) * 3)` print, and why is it not 15?
- **10.2** What does `float("2.50") * 2` give, and what type is it?

See also. Sheet 1, exercise 4. The Function reference, Week 1: `int`, `float`, `str` and `bool`.

Where the café is now

Slides, pages 44 to 47.

The café is a runnable file, `cafe_w1.py`: a name, an open sign, three prices, one hard-coded order, a printed receipt, and one deliberately ugly total as a souvenir of the float wall. Next week it learns to listen and remember: keyboard input so customers can order for themselves, formatting so we finally say goodbye to `9.600000000000001`, its first functions, and saving the day's revenue to a file that survives the shop closing for the night.

> **In the project.** The project is software like this café, grown over the term: from week 7 your group builds a trading desk for a fictional fund, out of the tools the lectures hand you week by week. Every week's notes point to where their tools come back. The project brief, in the Project module that opens on Friday of week 5, is the place to start.

What the exam can ask from this week, as page 45 lists it: state the output of `print`, arithmetic and conversion lines (the precedence traps, `bool("False")`, string `*` and `+`); explain float imprecision with `0.1 + 0.1 + 0.1` in one sentence; spot the error in `Print(...)`, in mismatched quotes, in

text joined to a number with `+`, and in `true`; and the theory: machine code against assembly against high-level languages, and what a string is underneath. You will read code and write what it prints, so practice reading before running.

The checklist on page 47 is your revision list: tick it honestly. Anything you cannot tick, read its section here again, then do the matching exercise on sheet 1; if it still does not click, at least you know exactly what to ask your tutor. The last box matters most: Python 3 and VS Code installed and working on your own machine. For a second voice, py4e's *Installing Python* and chapters 1 and 2, with their videos and auto-graded exercises, or Automate the Boring Stuff's introduction and chapter 1. A few of their exercises use `input()`, which is week 2, and Automate the Boring Stuff's chapter 1 also uses `round()`, which waits until week 4: park those, or peek ahead.

Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Cover the output under each `>>>` block in sections 5 to 10, and the pictures beside them, write what Python prints, then look. Write it exactly: `7` and `7.0` are different answers.
- Redo the precedence table in section 6 with its "Python says" column and the picture under it covered. Then change one number in each row and work out the new result on paper before you check it in Python.
- Explain, in one sentence each and without looking, why `0.1 + 0.1 + 0.1` is not `0.3`, what a string is underneath, and what Python does with a value no name points at any more; then compare with sections 7, 4 and 9.
- Name the four types of section 10 and the conversion function for each, from memory.
- In a few days, answer the check-yourself questions again without these notes, then take the week's self-check by hand, with Python closed.

Answers to check yourself

- **1.1** Hardware is the physical machine: processor, memory, disks, screen. Software is the instructions that make the hardware do something useful.
- **2.1** `110` is one 4, one 2 and no 1: six. Seven is `111`: 4 + 2 + 1.
- **2.2** The top rung: Swift is a high-level language, like Python. Below it sit assembly and machine code, and a compiler or an interpreter turns it into machine code, the only thing the processor runs.
- **3.1** The apostrophe in `It's` closes the string early, so the rest is not valid Python. Wrap the text in double quotes: `print("It's open")`.
- **3.2** Nothing but the error: `SyntaxError: '(' was never closed`. Python reads the whole file before it runs any line, so even `ready` never prints.
- **4.1** `<class 'str'>`: the quotes make it text, whatever the characters inside them.
- **5.1** 8: `Tab` is 3, the tab is one character, written with two keystrokes, and `here` is 4.
- **5.2** `Hi!Bye` on one line: the first `print` ends with `!` instead of a new line.
- **5.3** Escape the apostrophe: `print('It\'s open')`.
- **6.1** `30 // 12` is 2 full boxes, and `30 % 12` leaves 6 muffins.
- **6.2** The program stops with a `ZeroDivisionError`. Check that the number of orders is not zero before dividing: week 3's `if` is how.

- **7.1** Yes: it prints `0.75` exactly. A half and a quarter have exact binary forms; a tenth does not, so `0.1 + 0.1 + 0.1` collects errors.
- **7.2** `10.0 ** 400` raises `OverflowError: (34, 'Result too large')`: a float has fixed limits. `10 ** 400` is an int, and ints are unbounded, so it prints all 401 digits.
- **7.3** `<class 'float'>`: an int mixed with a float gives a float.
- **8.1** Both are `True`: a space and the character `0` are non-empty strings. Only `""` is `False`.
- **8.2** `True` is a keyword, a value Python keeps for itself, so you cannot assign to it. `true` is an ordinary name, so Python accepts it, and you will confuse yourself later.
- **9.1** Still `10.0`. `total` was computed from the value `qty` had then; rebinding `qty` does not reach back.
- **9.2** `price_2` and `Class` are legal. `2nd_price` starts with a digit, and `class` is a keyword.
- **10.1** `555`: `str(5)` is the text `"5"`, and `*` repeats text three times.
- **10.2** `5.0`, a `float`: `float("2.50")` turns the text into the number 2.5.